MATLAB Serial/UDP Datalogger

Student Guide

The datalogger receives binary packets from a device, displays signals in real time, and exports samples to the MATLAB workspace. Two applications are available: `serialDataLoggerApp` for serial communication and `udpDataLoggerApp` for UDP. The **Packet**, **Log**, and **Plots** tabs work in the same way in both applications.

1 Installation and startup

1.1 On your own computer

Download the datalogger archive from Moodle and extract it to a permanent folder. In MATLAB, open the `DataLoggerApp` folder containing `setup.m`, then run:

```
setup
```

This command adds the required folders to the MATLAB path. The configuration is not saved permanently: repeat this step in each new MATLAB session. The UDP version also requires the *Instrument Control Toolbox*, as stated in the software distribution email.

1.2 On laboratory computers

A copy of the application is already available in the shared folder on the laboratory servers. You do not need to extract a personal copy. In MATLAB, open the folder containing the supplied `dataloggerSetup.m` script and run:

```
dataloggerSetup
```

The script uses the path `X:\LabControlli\MATLAB\Utils\DataLogger\setup`. Run it in each new session. If the path is unavailable, check that the shared `X:` drive is mounted.

1.3 Opening the application

Run either command:

```
serialDataLoggerApp
% or:
udpDataLoggerApp
```

Opening the window does not start the connection. Configure the connection and packet before setting the switch to **On**. The settings must match the program running on the device and the instructions for your lab exercise.

2 Measurement workflow

1. Connect and prepare the device that will transmit the data.
2. Open the serial or UDP app and configure the **Connection** tab.

3. In **Packet**, specify the type, dimension, and order of the received signals.
4. In **Log**, set the sampling period, buffer capacity, and export variable name.
5. In **Plots**, select the signals to display.
6. Set the switch to **On**. The green **Connected** indicator means that the port has opened; also check that samples are arriving.
7. At the end of the measurement, set the switch to **Off**, click **Save to workspace**, and save the variable to a MAT file.

The **Connection** and **Packet** settings cannot be changed while connected. Disconnect the app before editing them.

3 Configuring the connection

3.1 Serial

In **Serial Port**, select the device port, for example `COM3` on Windows. In **Baudrate**, select the same speed as the device; the default is 115200 baud. Close any serial monitors or other applications using the port. If you connected the device after opening the app and its port does not appear, reopen the application.

3.2 UDP

In **Local UDP Port**, enter the port on the PC to which the device sends data. Configure the device with the PC's IP address and that same destination port. Use an explicit, available port, such as 5000 if specified in the lab exercise. The default value of 0 lets the system choose a port and is unsuitable when the destination must be fixed in advance.

The **Remote IP Address** and **Remote UDP Port** fields are used to send optional start and stop commands to the device. They do not configure the destination of the data sent by the device or select which sender's data to accept. The app uses IPv4.

3.3 Optional start and stop commands

Enable **Send Transmission Start/Stop Commands** only if the device program supports them. The app sends the **Start byte** when connecting and the **Stop byte** when disconnecting. These are single bytes with integer values between 0 and 255, rather than text strings. The defaults are 65 and 66 (ASCII characters A and B). For UDP, also specify the remote IP address and port that receive commands. To end a trial that uses these commands, explicitly set the switch to **Off** before closing the window.

4 Describing the received packet

Each row in **Packet structure** represents a signal. Set its **Signal Type** and **Dimension**: a scalar has dimension 1, and a three-component vector has dimension 3. The table's context menu lets you add, delete, and reorder rows (**Add row**, **Delete row**, **Move up**, **Move down**).

Type	Bytes per component
`int8`, `uint8`	1
`int16`, `uint16`	2
`int32`, `uint32`, `single`	4
`double`	8

For example, a packet containing a `uint32` counter and two `single` measurements can be described by two rows: `uint32` with dimension 1, and `single` with dimension 2. It contains 12 data bytes before encoding and termination. If you use three scalar rows instead, the two measurements will also be separate signals in the exported data.

The type, order, and dimension must match the transmitted data. Values must be binary, in *little-endian* order, stored consecutively without padding between fields. A stream of text lines, such as comma-separated numbers, does not match this format.

The **COBS Encoded** and **Null terminated** options must match the transmitter's format, including when using UDP:

- Both enabled: each packet is COBS-encoded and followed by a zero byte. This is the default configuration.
- Both disabled: raw binary data, with a fixed packet length determined by the table.
- Only COBS enabled: encoded packets of fixed length; allowed only for packets containing fewer than 254 data bytes.

Zero termination without COBS is not allowed. To use the app, you only need to know the format used by the device; you do not need to implement the encoding in MATLAB.

5 Recording and exporting data

Configure the following fields in the **Log** tab:

Tx Sample Time [s] The period T_s at which the device produces one packet. For transmission at 100 Hz, enter 0.01. This field determines the time assigned to samples: it does not change the device's sampling rate or measure when data arrive.

Buffer size The maximum number of packets retained, from 1 to 10^6. Once the buffer is full, new samples replace the oldest ones. For a trial lasting D, choose at least $\lceil D/T_s \rceil$ samples, with a suitable margin.

Log to Workspace Enables the workspace export options.

Variable name The MATLAB variable name, initially `data`. Exporting again with the same name overwrites the previous variable.

Save to workspace Exports a snapshot of the data currently retained. It does not create a file on disk or clear the buffer.

Save to workspace on exit Exports when the window closes, provided **Log to Workspace** is also enabled.

Clear buffer Clears the samples and displayed points. Save any data you want to keep first.

The time axis advances by T_s for each accepted packet. At the start of the session, the first sample has index 1 and time T_s. This time does not reveal packet loss. If you need to detect missing packets, include a counter or timestamp transmitted by the device among the signals. To compare trials with a common time origin, subtract the time of the first exported sample.

After clicking **Save to workspace**, the exported structure contains:

Field	Meaning
`data.sampleIndex`	Vector of indices of the retained samples.
`data.time`	Vector of reconstructed times, in seconds.
`data.signals{k}`	Signal matrix for row k in **Packet**: components are rows and samples are columns.
`data.size`	Number of exported samples.

For the example packet (a counter and a vector of two measurements):

```
% After exporting to the variable data:
assert(data.size > 0, 'No samples acquired');
t = data.time - data.time(1);
counter = data.signals{1}(1,:);
measurement1 = data.signals{2}(1,:);
measurement2 = data.signals{2}(2,:);
figure;
plot(t, measurement1, t, measurement2);
xlabel('Time [s]'); grid on;
legend('Measurement 1', 'Measurement 2');
save('trial01.mat', 'data');
```

Keep the MAT file for subsequent analysis.

6 Displaying signals

In **Plots**, **Plot grid (# rows/cols)** sets the number of rows and columns in the plot grid. Expand the plot tree and select the signal components to display in each plot. Multiple components can share the same axes.

In **X Axis**, select the unit (time or **Sample #**) and **Duration**, the width of the displayed window in the selected unit. In **Y Axis**, keep **Auto** enabled for automatic scaling, or set **YMin** and **YMax**.

Refresh every # packets received sets how many packets are received between display updates. Higher values reduce the update rate without subsampling the exported data. For example, with $T_s = 0.01$ s and a value of 10, the display updates approximately every 0.1 s for a steady stream. Context menus let you change curve colours and styles and show the legend.

7 Essential API: saving a repeatable configuration

To avoid configuring the interface each time, save a MATLAB script that opens the app with your trial settings. Parameters are optional; the connection is still started using the switch in the app. The following examples use illustrative values that you must adapt to your device.

7.1 Serial example

```
app = serialDataLoggerApp( ...
    serialPortName="COM3", baudRate=115200, ...
    packetSpec={'uint32','2*single'}, ...
    enableCOBS=true, enableNullTerminator=true, ...
    txSampleTime=0.01, bufferSize=10000, ...
    enableLogToWorkspace=true, enableSaveOnExit=true, ...
    varName="data", ...
    plotsGridSize=[2 1], ...
    plotsContentList=[1 2 1; 2 2 2], ...
    plotsDuration=5, plotsRefreshRatio=10);
```

`packetSpec` is a cell array: each entry describes a row of the **Packet** table. The notation `'2*single'` means two components of type `single`.

Each row of `plotsContentList` contains `[plotNumber signalNumber componentNumber]`, with indices starting at 1. In this example, the first plot shows the first component of the second signal, and the second plot shows its second component. `plotsDuration` sets the initial plot duration in seconds.

7.2 UDP example

```
app = udpDataLoggerApp( ...
    localUdpPort=5000, ...
    packetSpec={'uint32','2*single'}, ...
    enableCOBS=true, enableNullTerminator=true, ...
    txSampleTime=0.01, bufferSize=10000, ...
    varName="data", ...
    plotsContentList=[1 2 1; 1 2 2], ...
    plotsDuration=5, plotsRefreshRatio=10);
```

The device must send data to the PC's IP address on port 5000. If start and stop commands are required, add `enableStartStopCmds=true`, `txStartCmd=65`, and `txStopCmd=66` to the call, replacing the command values as needed. For UDP, also add `remoteIpAddress="192.168.1.50"` and `remoteUdpPort=5001`, using the device's actual command address and port. The same command parameters, except for the remote IP address and port, are available in the serial app.

8 Troubleshooting

MATLAB does not recognise the app command. Run `setup` again from the correct folder on your own computer, or `dataloggerSetup` in the laboratory.

The serial port does not open. Check the cable, selected port, and any programs already using it. Make sure the operating system recognises the device.

The connection is active, but no data arrive. The green indicator confirms that the port is open. Check that the device is transmitting, any commands are correct, and the packet format and encoding match. For UDP, check the PC's IP address, local port, network connection, and MATLAB's firewall permissions.

Values or times look incorrect. Check signal order, types, dimensions, binary format, and **Tx Sample Time [s]**.

The first samples of the trial are missing. The buffer capacity may have been exceeded. Increase it before the measurement. Exporting cannot recover samples that have already been replaced.

The measurement is missing from the workspace. Click **Save to workspace** and check **Variable name**. Then explicitly save the data to disk using `save`.